RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF INFORMATION TECHNOLOGY

CS23621 - MOBILE APPLICATION AND DEVELOPMENT LABORATORY

LABORATORY RECORD

NAME : GOWTHAM S

UNIVERSITY REG NO : 2116231001050

YEAR / BRANCH : III / IT

SEMESTER : VI

ACADEMIC YEAR : 2025-26

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: GOWTHAM S

Academic Year: 2025-26 **Semester:** VI **Branch:** B.Tech - IT

Register No. 2116231001050

Certified that this is the Bonafide record of work done by the above student in the CS23621- Mobile Application Development Laboratory ***Laboratory during the academic year 2026 – 2027.***

Signature of Faculty in-charge

Submitted for the Practical Examination held on 13/05/2026

Internal Examiner External Examiner

INDEX

EXPT NO.	EXPERIMENT NAME	GITHUB QR
1	Design a simple Android app using Kotlin basics to display a welcome message using variables, data types, and control statements.	
2	Create an Android app to perform arithmetic operations using functions for addition, subtraction, multiplication, and division.	
3	Develop an Android app to manage student details using classes and objects in Kotlin.	
4	Build your first Android app with a button and handle click events to display a toast message.	
5	Design a login screen using LinearLayout or ConstraintLayout with username and password fields.	
6	Create navigation between Login and Home screens using intents or navigation components.	
7	Demonstrate Activity and Fragment lifecycle methods and display lifecycle changes on the screen.	
8	Develop an Android app using MVVM architecture with ViewModel and display a list of items.	
9	Build an Android app with Room database for data persistence to store and retrieve data.	
10	Create an Android app using RecyclerView to display a list of items with images and click events.	
11	Develop an Android app to fetch data from an API and display the fetched data.	
12	Implement Repository Pattern and WorkManager for efficient background data fetching and caching.	
13	Design a user-friendly Android app UI with proper colors, spacing, alignment, and usability principles.	

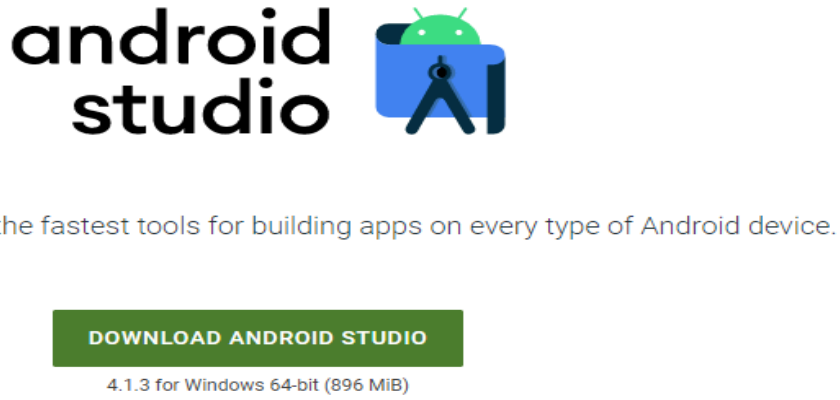
Ex. Nos. 1		INSTALLATION OF ANDROID STUDIO
Date :		

AIM: To install Android Studio to develop Mobile Applications using Kotlin.

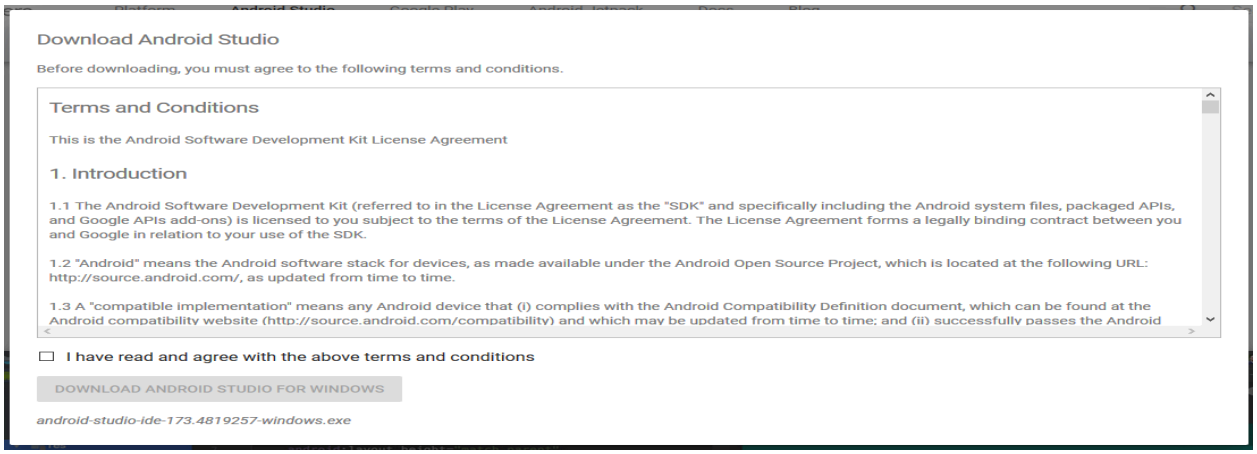
STEPS TO INSTALL ANDROID STUDIO:

Step 1: Head over to **https://developer.android.com/studio/#downloads** to get the Android Studio executable or zip file.

Step 2: Click on the **Download Android Studio** Button.



Click on the "I have read and agree with the above terms and conditions" checkbox followed by the download button.

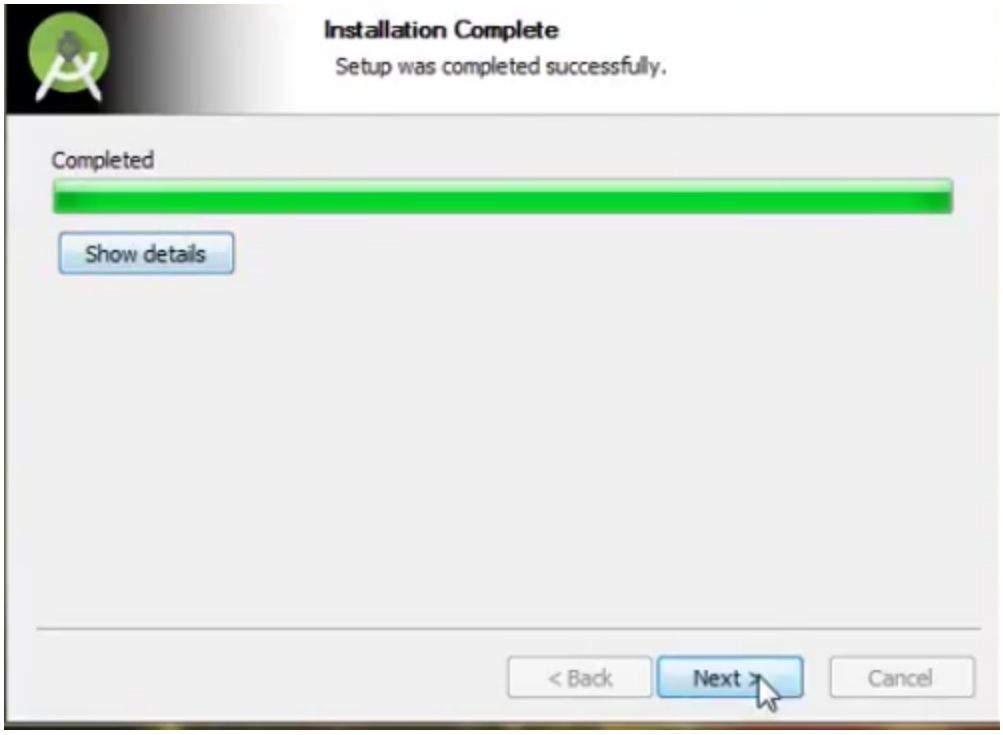


Click on the Save file button in the appeared prompt box and the file will start downloading.

Step 3: After the downloading has finished, open the file from downloads and run it. It will prompt the following dialog box.

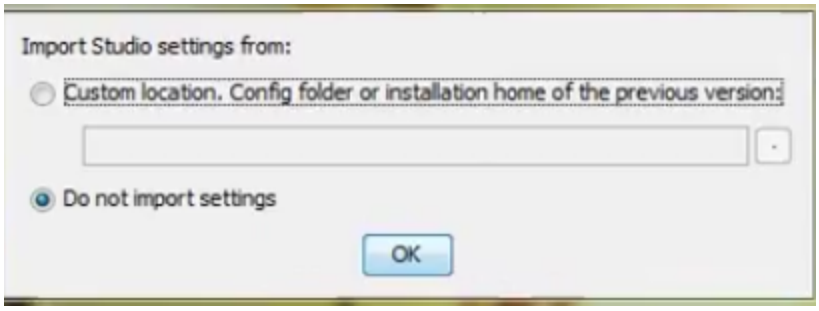
Click on next. In the next prompt, it'll ask for a path for installation. Choose a path and hit next.

Step 4: It will start the installation, and once it is completed, it will be like the image shown below.



Click on next.

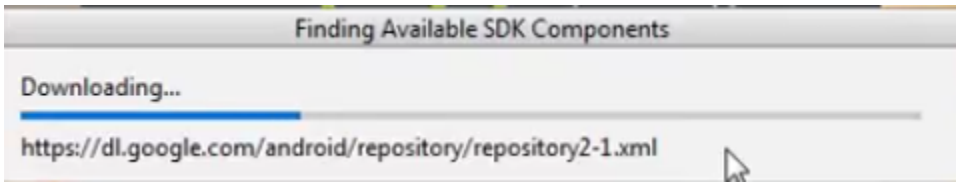
Step 5: Once " **Finish** " is clicked, it will ask whether the previous settings need to be imported [if the android studio had been installed earlier], or not. It is better to choose the 'Don't import Settings option'.



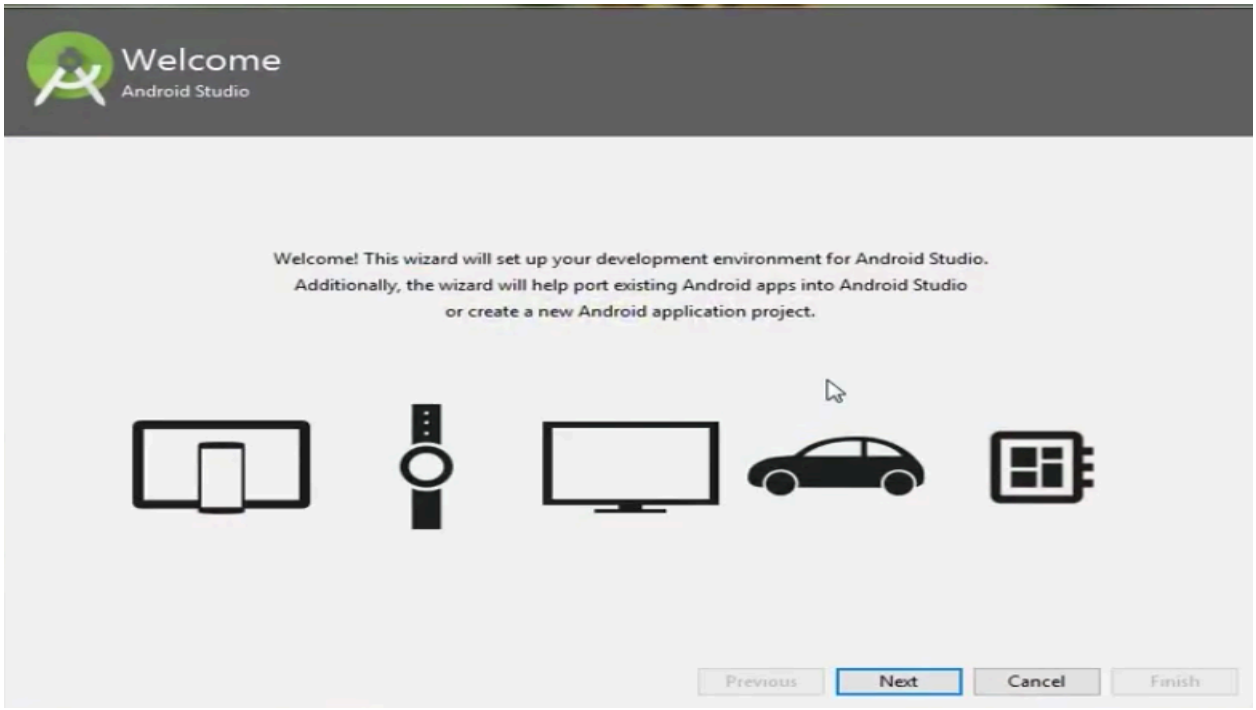
Click the **OK** button.

Step 6: This will start the Android Studio.

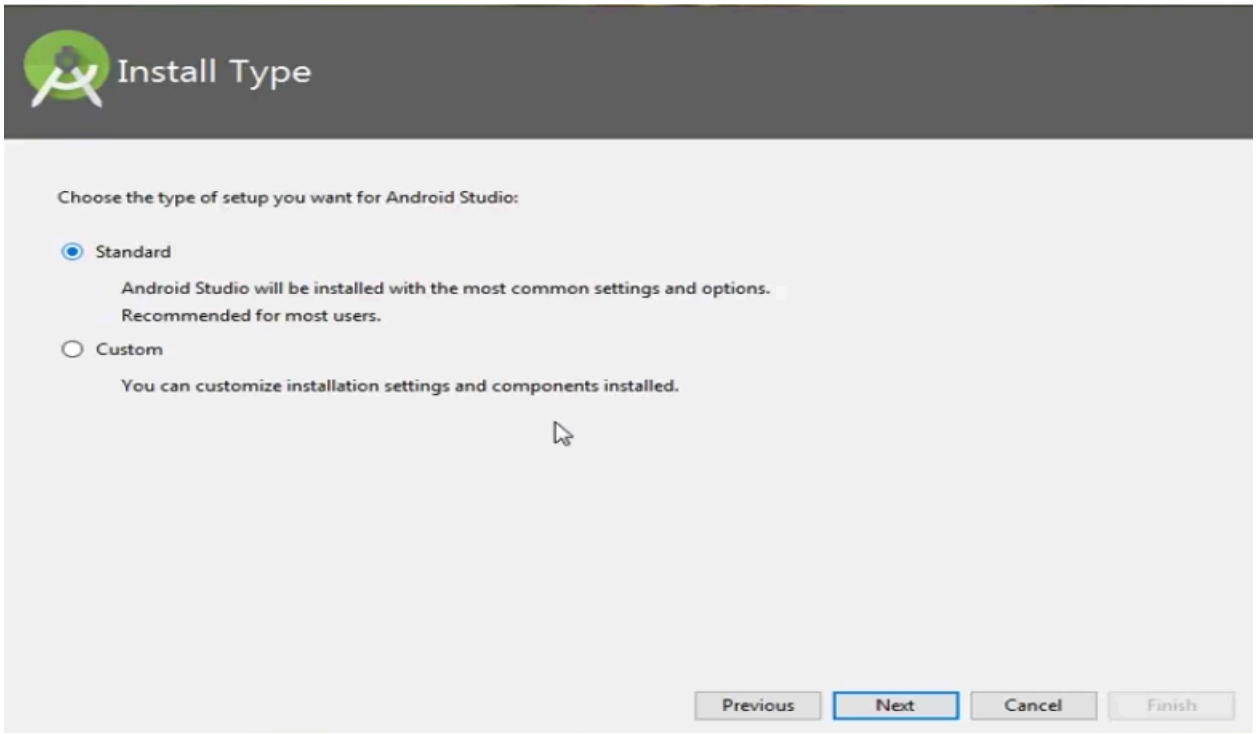
Meanwhile, it will be finding the available SDK components.



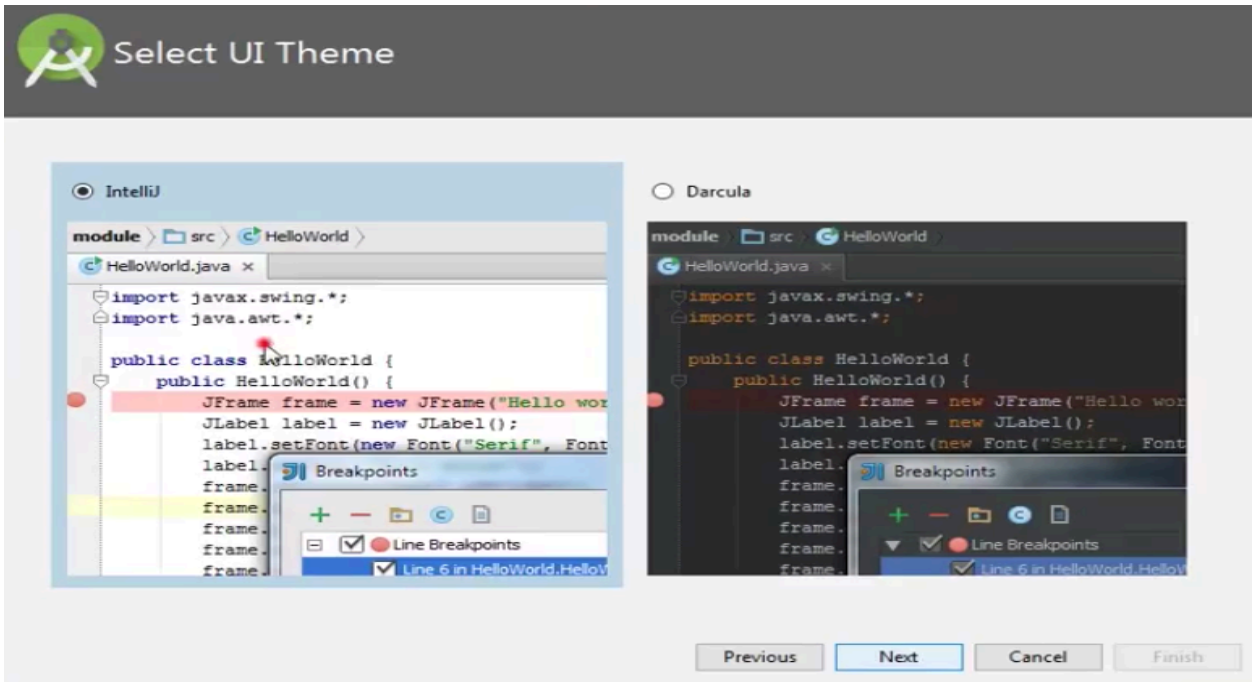
Step 7: After it has found the SDK components, it will redirect to the Welcome dialog box.



Click on **Next** .

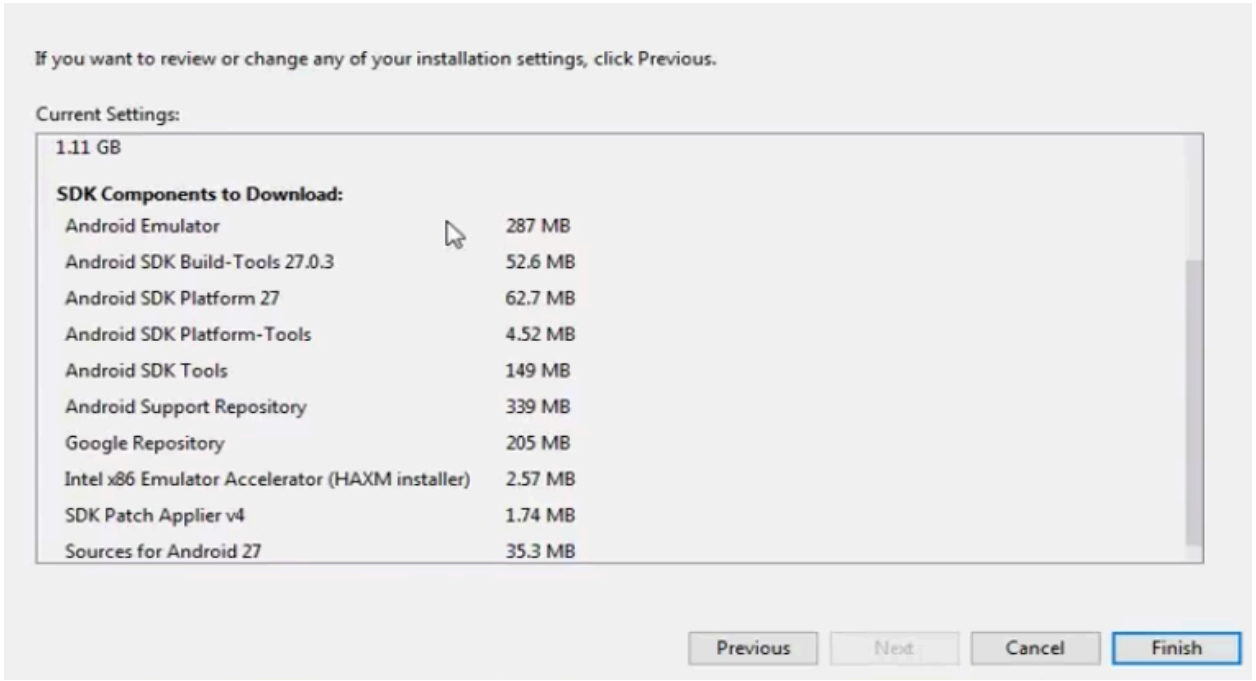


Choose Standard and click on Next. Now choose the theme, whether the **Light** theme or the **Dark** one. The light one is called the **IntelliJ** theme whereas the dark theme is called **Dracula** . Choose as required.

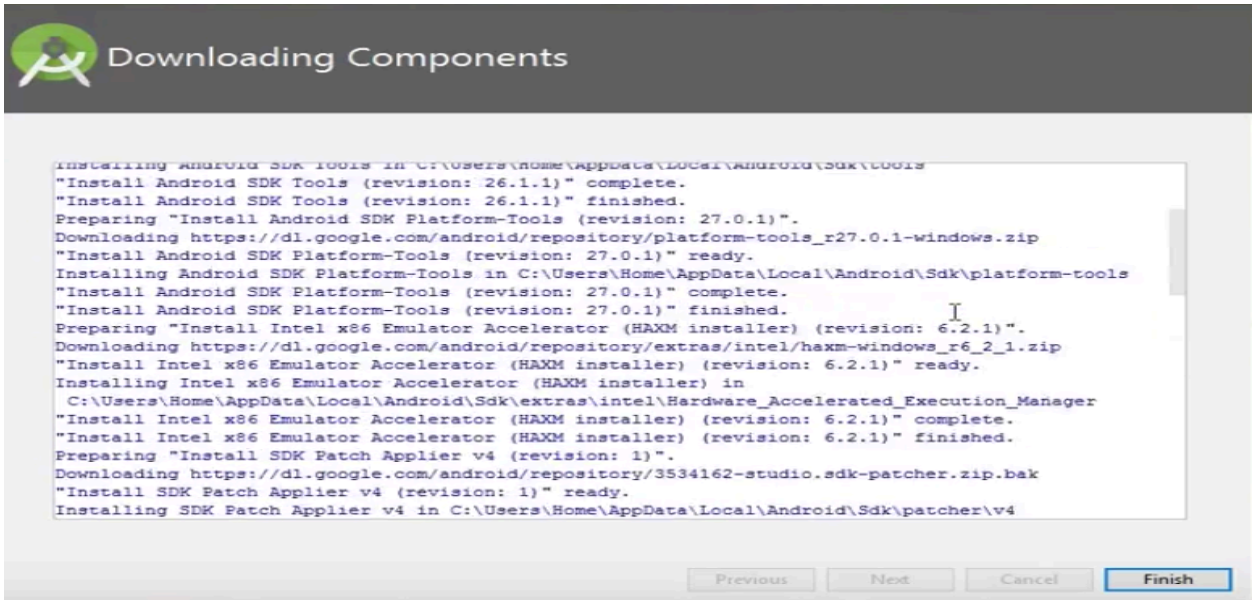


Click on the **Next** button.

Step 8: Now it is time to download the SDK components.

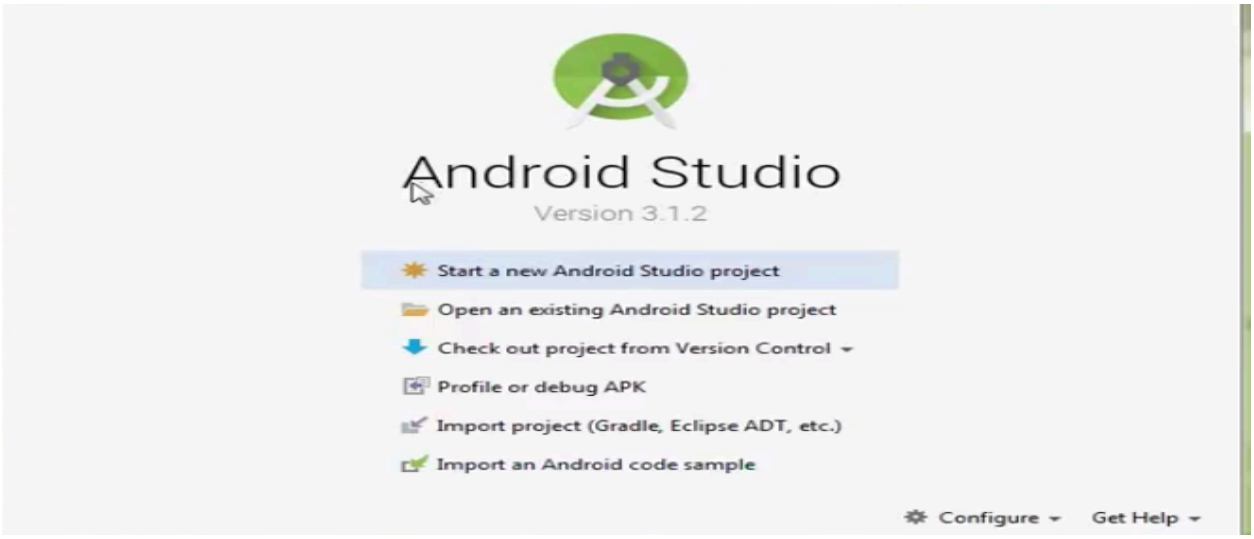


Click on Finish. Components begin to download let it complete.



The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

Step 9: Click on **Start a new Android Studio project** to build a new app.



RESULT: Thus, the Android was successfully installed and configured for Android App Development using Kotlin.

Ex. Nos. 2		**INTRODUCTION TO KOTLIN**

2(a) Kotlin Basics

AIM: To study and implement the basic concepts of **Kotlin programming language** including variables, data types, operators, control statements.

THEORY

Kotlin is a modern, statically typed programming language developed by **JetBrains**. It is officially supported for **Android app development** and runs on the **Java Virtual Machine (JVM)**. Kotlin is designed to be concise, safe, and fully interoperable with Java.

Key Features of Kotlin

- Simple and concise syntax
- Null safety (reduces NullPointerException)

- Fully interoperable with Java
- Supports object-oriented and functional programming
- Less boilerplate code compared to Java

Basic Concepts in Kotlin

1. **Variables**
    - val → immutable (read-only)
    - var → mutable
2. **Data Types**
    - Int, Double, Float, Char, String, Boolean
3. **Operators**
    - Arithmetic, Relational, Logical operators
4. **Control Statements**
    - if, when, for, while

CODE:

```
// Kotlin Basics Program

fun main() {
    // Variables
    val language: String = "Kotlin"
    var age: Int = 10

    println("Language Name: $language")
    println("Age: $age")
```

```
// Arithmetic Operations
val a = 10
val b = 5

println("Addition: ${a + b}")
println("Subtraction: ${a - b}")
println("Multiplication: ${a * b}")
println("Division: ${a / b}")
```

```
// if-else statement
   if (age >= 18) {
      println("Eligible to vote")
   } else {
      println("Not eligible to vote")
   }

   // when statement
   val day = 3
   when (day) {
      1 -> println("Monday")
      2 -> println("Tuesday")
      3 -> println("Wednesday")
      4 -> println("Thursday")
      5 -> println("Friday")
      else -> println("Weekend")
   }

   // for loop
   println("For loop output:")
   for (i in 1..5) {
      println(i)
   }
```

```
 // while loop
    println("While loop output:")
    var i = 1
    while (i <= 5) {
        println(i)
        i++
    }
}
```

OUTPUT

Language Name: Kotlin

Age: 10

Addition: 15

Subtraction: 5

Multiplication: 50

Division: 2

Not eligible to vote

Wednesday

For loop output:

1

2

3

4

5

While loop output:

1

2

3

4

5

RESULT: Thus, the basic concepts of **Kotlin programming language** such as variables, operators, control statements were successfully studied and implemented.

2(b) Kotlin Functions

AIM: To study and implement **functions in Kotlin** including function definition, function call, functions with parameters, and functions with return values.

THEORY

A **function** in Kotlin is a block of code that performs a specific task. Functions help in **code reusability**, **better readability**, and **modular programming**.

In Kotlin:

- Every program starts execution from the main() function
- Functions are defined using the keyword fun

Advantages of Functions

- Avoids repetition of code
- Improves program structure
- Makes debugging easier

TYPES OF FUNCTIONS USED

1. **Function without parameters and without return value**
2. **Function with parameters**
3. **Function with return value**

CODE:

```
// Kotlin Functions Program

fun main() {

    // Function without parameters
    greet()

    // Function with parameters
    addNumbers(10, 5)

    // Function with return value
    val result = multiply(4, 3)
    println("Multiplication Result: $result")
}

// Function without parameters and return value
fun greet() {
    println("Welcome to Kotlin Functions")
}

// Function with parameters
fun addNumbers(a: Int, b: Int) {
    val sum = a + b
    println("Sum: $sum")
}

// Function with return value
fun multiply(a: Int, b: Int): Int {
    return a * b
}
```

OUTPUT

```
Welcome to Kotlin Functions
Sum: 15
Multiplication Result: 12
```

RESULT: Thus, the concept of **functions in Kotlin** was successfully studied and implemented using functions with and without parameters and return values.

2(c) Kotlin Classes and Objects

AIM: To study and implement **classes and objects in Kotlin** using class definition, properties, member functions, and object creation.

THEORY

Kotlin supports **Object-Oriented Programming (OOP)**.
A **class** is a blueprint used to create objects.
An **object** is an instance of a class.

In Kotlin:

- Classes are defined using the keyword class
- Objects are created using the class name
- Constructors are used to initialize class properties

Advantages of Classes and Objects

- Helps in data abstraction
- Improves code organization
- Supports real-world modeling

CONCEPTS USED

1. Class definition
2. Primary constructor
3. Member variables (properties)
4. Member functions
5. Object creation

CODE:

```
// Kotlin Classes and Objects Program

fun main() {

    // Creating object of Student class
    val s1 = Student("John", 21)

    // Calling member function
    s1.display()
}

// Class definition
class Student(var name: String, var age: Int) {

    // Member function
    fun display() {
        println("Student Name: $name")
        println("Student Age: $age")
    }
}
```

OUTPUT:

```
Student Name: John
Student Age: 21
```

RESULT: Thus, the concept of **classes and objects in Kotlin** was successfully studied and implemented.

Ex. Nos. 13		GUI COMPONENTS
Date :		

AIM: To design and implement a simple Android application using Jetpack Compose that demonstrates basic GUI components and allows dynamic changes to font size, font color, and background color through button interactions.

CODE:

//MainActivity.kt

```
package com.example.guibasics

import android.os.Bundle

import androidx.activity.ComponentActivity

import androidx.activity.compose.setContent

import androidx.activity.enableEdgeToEdge

import androidx.compose.foundation.layout.fillMaxSize

import androidx.compose.foundation.layout.padding

import androidx.compose.material3.Scaffold

import androidx.compose.runtime.Composable

import androidx.compose.ui.Modifier

import androidx.compose.ui.tooling.preview.Preview

import com.example.guibasics.ui.theme.GuiBasicsTheme


class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        enableEdgeToEdge()

        setContent {

            GuiBasicsTheme {

                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->

                    GuiApp(
```

```
                    modifier = Modifier.padding(innerPadding)
                )
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    GuiBasicsTheme {
        GuiApp()
    }
}
```

//GuiApp.kt

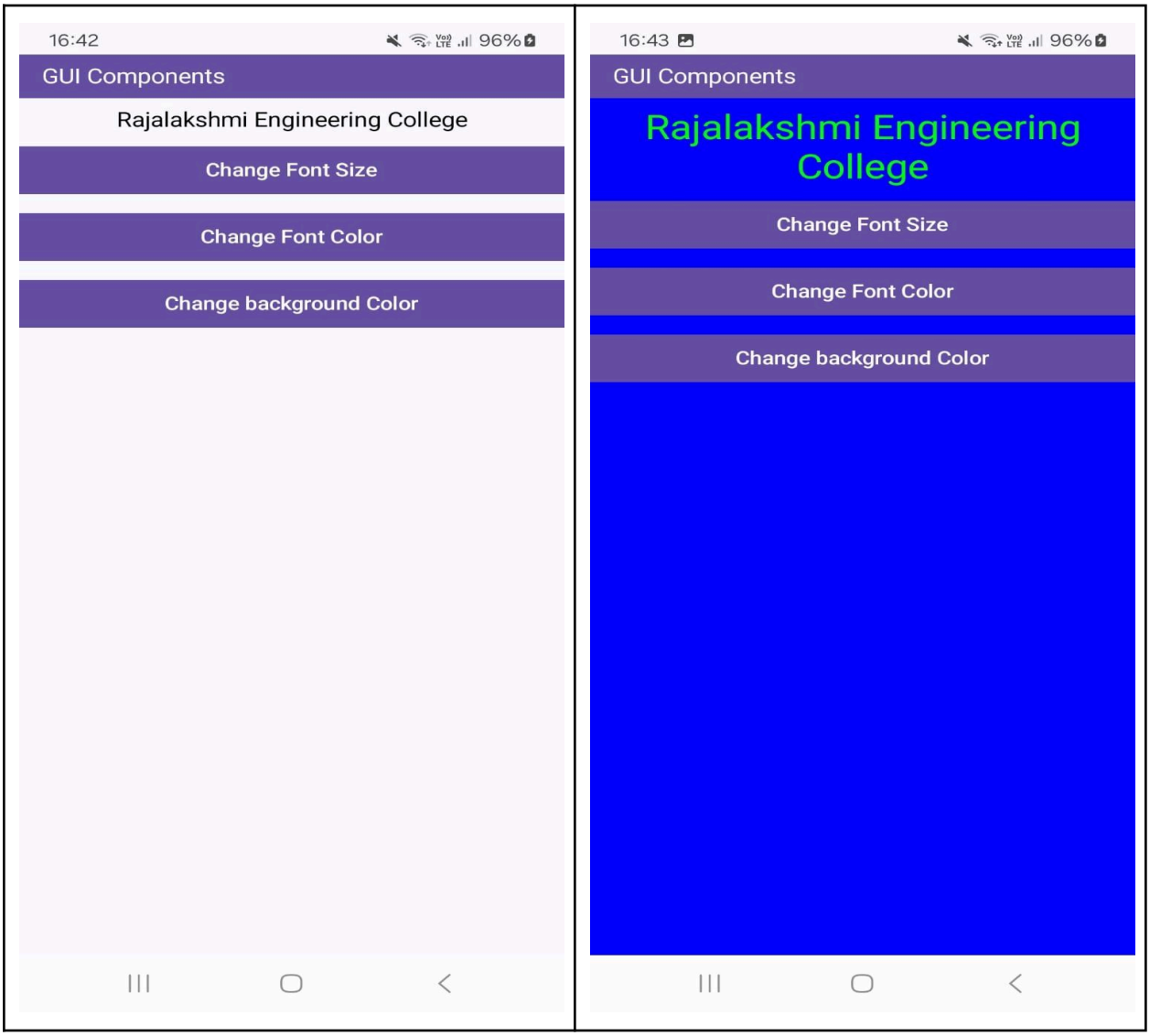
```
package com.example.guibasics

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.RectangleShape
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.guibasics.ui.theme.Purple40

@Composable
fun GuiApp(modifier: Modifier = Modifier){

    val defaultBg = MaterialTheme.colorScheme.background

    val sizes = listOf(16, 20, 24, 28, 32)
    val colors = listOf(Color.Black, Color.Blue, Color.Green)
    val bgs = listOf(defaultBg, Color.Blue, Color.Green)

    var fontsize by remember { mutableStateOf(16) }
    var fontcolor by remember { mutableStateOf(Color.Black) }
    var bg by remember { mutableStateOf(defaultBg) }

    var fsc by remember { mutableStateOf(0) }
```

```
var fcs by remember { mutableStateOf(0) }
var bgc by remember { mutableStateOf(0) }

fun changeFontSize(){
    fontsize = sizes[(fsc + 1) % sizes.size]
    fsc++
}

fun changeFontColor(){
    fontcolor = colors[(fcs + 1) % colors.size]
    fcs++
}

fun changeBg(){
    bg = bgs[(bgc + 1) % bgs.size]
    bgc++
}

Column(
    modifier = modifier
        .background(bg)
        .fillMaxSize()
        .padding(12.dp)
) {

    Text(
        text = "GUI Components",
        modifier = Modifier
            .background(Purple40)
            .padding(vertical = 8.dp, horizontal = 16.dp)
```

```
        .fillMaxWidth(),
    style = TextStyle(fontSize = 16.sp, color = Color.White)
)

Spacer(modifier = Modifier.height(8.dp))

Text(
    text = "Rajalakshmi Engineering College",
    style = TextStyle(fontSize = fontsize.sp, color = fontcolor),
    textAlign = TextAlign.Center,
    modifier = Modifier.fillMaxWidth()
)

Spacer(modifier = Modifier.height(8.dp))

Button(
    onClick = { changeFontSize() },
    modifier = Modifier.fillMaxWidth(),
    colors = ButtonDefaults.buttonColors(containerColor = Purple40),
    shape = RectangleShape
) {
    Text("Change Font Size")
}

Spacer(modifier = Modifier.height(8.dp))

Button(
    onClick = { changeFontColor() },
    modifier = Modifier.fillMaxWidth(),
    colors = ButtonDefaults.buttonColors(containerColor = Purple40),
```

```
            shape = RectangleShape
        ) {
            Text("Change Font Color")
        }

        Spacer(modifier = Modifier.height(8.dp))

        Button(
            onClick = { changeBg() },
            modifier = Modifier.fillMaxWidth(),
            colors = ButtonDefaults.buttonColors(containerColor = Purple40),
            shape = RectangleShape
        ) {
            Text("Change Background Color")
        }
    }
}
```

OUTPUT:



RESULT: Thus, a simple Android application using Jetpack Compose was successfully developed to demonstrate GUI components and dynamic UI changes such as font size, font color, and background color based on user interaction.

Ex. Nos. 2		BASIC CALCULATOR

AIM: To design and implement a Simple Calculator Android application using Jetpack Compose that performs basic arithmetic operations (+, −, ×, ÷) and displays the expression and result based on user input.

CODE:

//MainActivity.kt

```
package com.example.calculator


import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.calculator.ui.theme.CalculatorTheme


class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CalculatorTheme {
```

```
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                CalculatorApp(
                    modifier = Modifier.padding(innerPadding)
                )
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    CalculatorApp()
}
```

//CalculatorApp.kt

```
package com.example.calculator
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.RectangleShape
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.calculator.ui.theme.Purple40
```

```
@Composable
fun CalculatorApp(modifier: Modifier = Modifier) {

    var current by remember { mutableStateOf("") }
    var expression by remember { mutableStateOf("") }
    var result by remember { mutableStateOf("") }

    var input1 by remember { mutableStateOf(0.0) }
    var operator by remember { mutableStateOf("") }
    var decimalUsed by remember { mutableStateOf(false) }

    fun executeClick(op: String) {

        if (op == "Clear") {
            current = ""
            expression = ""
            result = ""
            input1 = 0.0
            operator = ""
            decimalUsed = false
            return
        }

        if (op.length == 1 && op[0].isDigit()) {
            current += op
            expression = current
            return
        }

        if (op == ".") {
```

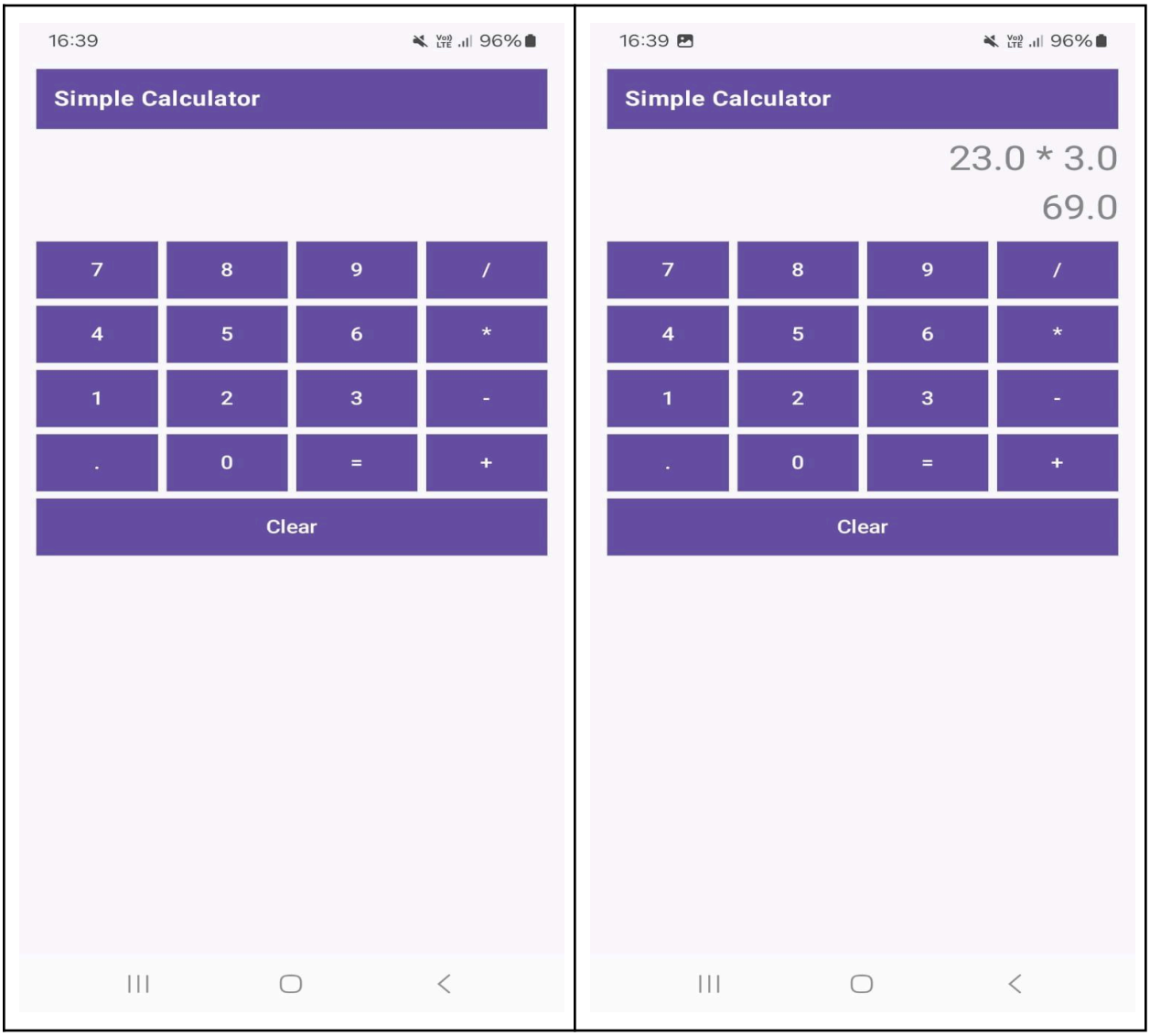
```
    if (!decimalUsed) {
        current = if (current.isEmpty()) "0." else current + "."
        decimalUsed = true
        expression = current
    }
    return
}

if (op in listOf("+", "-", "*", "/")) {
    if (current.isNotEmpty()) {
        input1 = current.toDouble()
        operator = op
        current = ""
        decimalUsed = false
        expression = op
    }
    return
}

if (op == "=") {
    if (current.isNotEmpty() && operator.isNotEmpty()) {
        val input2 = current.toDouble()
        val ans = when (operator) {
            "+" -> input1 + input2
            "-" -> input1 - input2
            "*" -> input1 * input2
            "/" -> input1 / input2
            else -> 0.0
        }
        result = ans.toString()
```

```
            expression = "$input1 $operator $input2"
            operator = ""
            current = ""
            decimalUsed = false
        }
        return
    }
}

Column(
    modifier = modifier
        .fillMaxSize()
        .padding(12.dp)
) {

    Text(
        text = "Simple Calculator",
        modifier = Modifier
            .fillMaxWidth()
            .background(color = Purple40)
            .padding(12.dp),
        color = Color.White,
        fontWeight = FontWeight.Bold,
        textAlign = TextAlign.Left
    )

    Text(
        text = expression,
        modifier = Modifier
            .fillMaxWidth()
```

```
            .padding(vertical = 8.dp),
        style = TextStyle(color = Color.Gray, fontSize = 28.sp),
        textAlign = TextAlign.End
    )

    Text(
        text = result,
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 12.dp),
        style = TextStyle(color = Color.Gray, fontSize = 28.sp),
        textAlign = TextAlign.End
    )

    Column(
        modifier = Modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(6.dp)
    ) {

        CalcRow(listOf("7", "8", "9", "/"), { executeClick(it) })
        CalcRow(listOf("4", "5", "6", "*"), { executeClick(it) })
        CalcRow(listOf("1", "2", "3", "-"), { executeClick(it) })
        CalcRow(listOf(".", "0", "=", "+"), { executeClick(it) })

        Button(
            onClick = { executeClick(op = "Clear") },
            modifier = Modifier
                .fillMaxWidth()
                .height(48.dp),
            shape = RectangleShape,
```

```
            colors = ButtonDefaults.buttonColors(containerColor = Purple40),
            contentPadding = PaddingValues(0.dp)
        ) {
            Text("Clear", color = Color.White)
        }
    }
  }
}

@Composable
private fun CalcRow(items: List<String>, onButtonClick: (String) -> Unit) {
  Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(6.dp)
  ) {
    items.forEach { label ->
      Button(
        onClick = { onButtonClick(label) },
        modifier = Modifier
          .weight(1f)
          .height(48.dp),
        shape = RectangleShape,
        colors = ButtonDefaults.buttonColors(containerColor = Purple40),
        contentPadding = PaddingValues(0.dp)
      ) {
        Text(label, color = Color.White)
      }
    }
  }
}
```

OUTPUT:



RESULT: Thus, the Simple Calculator application was successfully developed using Jetpack Compose, and it performed basic arithmetic operations with correct display of expression and result.

Ex. Nos. 3		ANDROID FRAGMENTS

```
package org.rajalakshmi.androidfragments


import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface {
                    // This calls the main container we define in the next file
                    StudentMainScreen()
                }
            }
        }
    }
}
```

```
//StudentMainScreen.kt
package org.rajalakshmi.androidfragments
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun StudentMainScreen() {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Android Fragments", color = Color.White) },
                colors = TopAppBarDefaults.topAppBarColors(containerColor = Color(0xFF6200EE))
            )
        }
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .padding(paddingValues)
                .fillMaxSize()
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
```

```
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(text = "Student Details", fontSize = 24.sp, color = Color.Gray)
            Spacer(modifier = Modifier.height(16.dp))

            // Calling the separate Composable files
            StudentBasicDetails()

            Spacer(modifier = Modifier.height(32.dp))

            StudentMarkDetails()
        }
    }
}
```

```
//StudentBasicDetails.kt
package org.rajalakshmi.androidfragments

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import org.rajalakshmi.androidfragments.components.CustomInputField

@Composable
fun StudentBasicDetails() {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(text = "Basic Details", fontSize = 20.sp, color = Color.Gray)

        CustomInputField(label = "Register No.", placeholder = "Register Number")
        CustomInputField(label = "Name", placeholder = "Name")
        CustomInputField(label = "Department", placeholder = "Department")
    }
}
```

```
//StudentBasicDetails.kt
package org.rajalakshmi.androidfragments

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import org.rajalakshmi.androidfragments.components.CustomInputField

@Composable
fun StudentMarkDetails() {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(text = "Mark Details", fontSize = 20.sp, color = Color.Gray)

        CustomInputField(label = "S.S.L.C.", placeholder = "S.S.L.C. Mark")
        CustomInputField(label = "H.Sc.", placeholder = "H.Sc. Mark")
        CustomInputField(label = "U.G.", placeholder = "U.G. C.G.P.A.")
    }
}
```